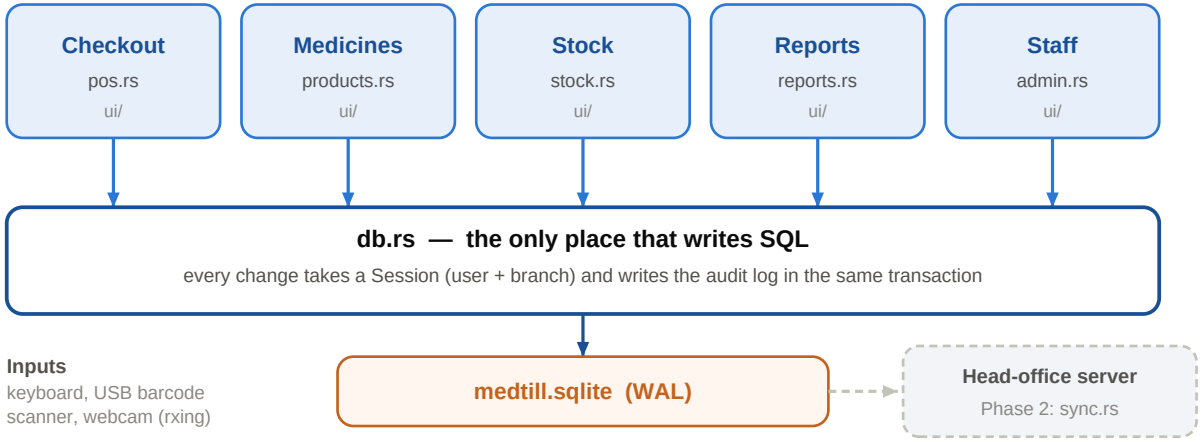
MedTill POS — how it fits together

One till = one app + one SQLite file. Every screen talks to the database only through db.rs.

The system

Screens (egui) — shown according to the logged-in role



Four rules that keep the data trustworthy

Stock is a ledger.
No stock number is stored. Every change is a row in stock_moves (+ received, – sold, – written off, ± counted, + voided). Stock = SUM(qty).

FEFO selling.
A sale takes units from the batch that expires first. Expired batches can't be sold.

Everyone is accountable.
Each change is made by a logged-in user and logged in audit.

Sync-ready ids.
Every id is a random 128-bit hex string, so branches never clash. Nothing is hard-deleted.

Roles and what they can open or do

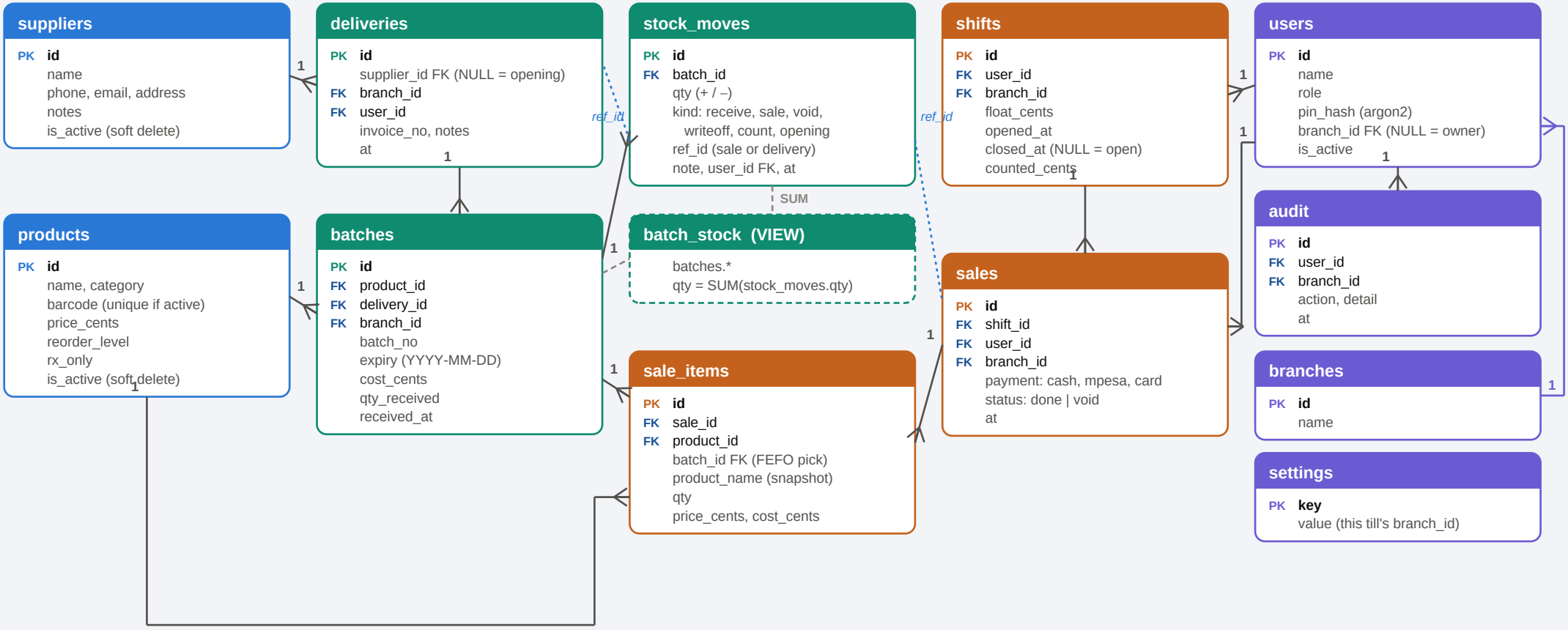
		Checkout (sell)	Sell Rx-only	Void sale	Stock: receive	Stock: write-off/count	Medicines & prices	Reports	Staff & settings
Owner	all branches	●	●	●	●	●	●	●	●
Branch manager	own branch	●	●	●	●	●	●	●	—
Pharmacist	own branch	●	●	—	●	—	—	—	—
Cashier	own branch	●	—	—	—	—	—	—	—

Permissions live in one place: Role::can(Action) in models.rs. The menu only shows screens the role may open; db.rs checks again (e.g. a cashier's sale with a prescription-only medicine is refused even if it reached the cart).

Login: pick your name, enter a 4–8 digit PIN (argon2-hashed). 5 wrong PINs pause logins for 30 s. The till locks itself after 5 minutes idle; a different person logging in starts with clean screens (never someone else's cart).

Entity-relationship diagram

Schema v2 (PRAGMA user_version = 2). PK = primary key, FK = foreign key. Lines read one → many.



Reading the diagram

People & place · Catalogue · Stock · Selling

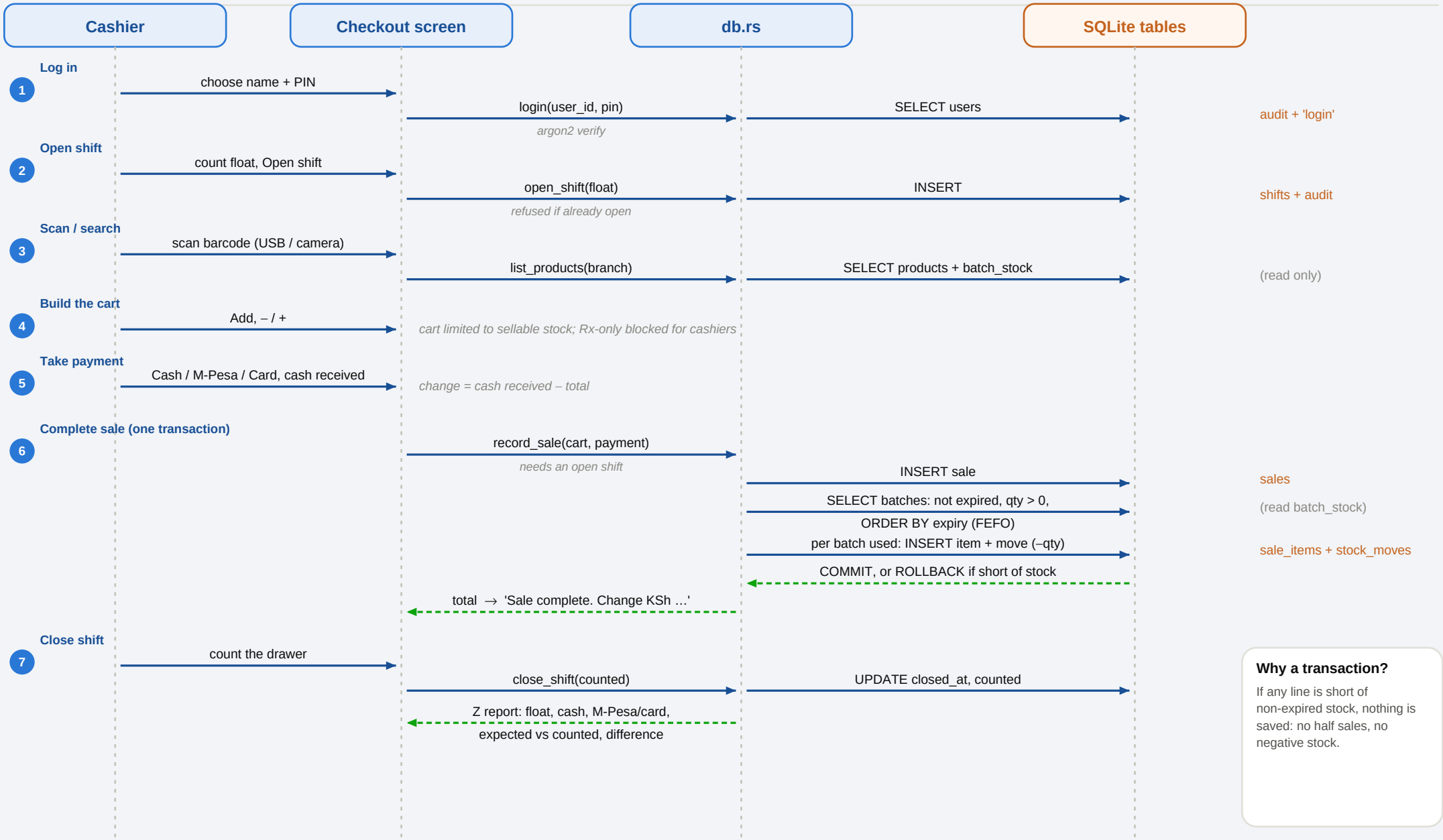
- branch_id is on every operational table (deliveries, batches, shifts, sales, audit) and points to branches.id — only users → branches is drawn.
- user_id is also on deliveries and stock_moves (who received / who adjusted).
- Dashed box = SQL view (computed, not stored). Dotted blue = stock_moves.ref_id: the sale or delivery that caused the move.
- sale_items snapshot name, price and cost, so old receipts and profit stay correct after price changes.
- One cart line can become several sale_items when it spans batches (FEFO). Old sales from before batches have batch_id = NULL.
- No FOREIGN KEY constraints on purpose: in Phase 2 sync, rows can arrive in any order. db.rs keeps integrity with transactions.

Indexes

products(barcode) unique, only active rows
batches(product_id, branch_id)
stock_moves(batch_id)
sales(branch_id, at)
sale_items(sale_id)
audit(at)

Interaction: selling a medicine

Sequence from login to the end-of-shift Z report. Arrows show calls; the right column shows what is written.



Why a transaction?

If any line is short of non-expired stock, nothing is saved: no half sales, no negative stock.

Interactions: stock, voids and access

Each flow: who → screen → db.rs function → tables written. Every write also adds an audit row.

Receive stock (a delivery)

Owner · Manager · Pharmacist

1. **Stock → Receive**
   Choose supplier (or 'None' for opening stock) and invoice no.
2. **Add lines**
   Scan the box or type the name; batch no., expiry (must be in the future), quantity, unit cost.
3. **Save delivery → db::receive**
   One transaction: 1 delivery row; per line 1 batch + 1 'receive' move (+qty).
4. **Result**
   Stock and value appear on the shelf list, earliest expiry first.

Writes: deliveries · batches · stock_moves · audit

Adjust a batch

Owner · Manager

1. **Stock → shelf → Adjust**
   Pick the batch (expired ones are red and default to 'Expired').
2. **Write off → db::write_off**
   Reason Expired / Damaged / Returned to supplier: a 'writeoff' move (−qty); can't remove more than is there.
3. **Stock count → db::count_stock**
   Enter what is physically on the shelf: a 'count' move for the difference (±).
4. **Result**
   Reports: expired value drops; audit shows who, what and why.

Writes: stock_moves · audit

Void a sale

Owner · Manager

1. **Reports → Sales (receipts)**
   Find the receipt (8-character number) and press Void.
2. **Give a reason → db::void_sale**
   One transaction: sale status → 'void'; every sale_item's units go back to the same batch with a 'void' move (+qty).
3. **Effect on money**
   Voided sales drop out of sales, profit and the shift's expected cash; the receipt stays visible as 'void'.
4. **A cashier made a mistake?**
   Lock → the manager logs in with their PIN → voids → locks again.

Writes: sales · stock_moves · audit

First start, login and staff

Everyone · Owner for staff

1. **First start → db::open / migrate**
   Upgrades an old database: products' stock → opening batches; old sales and deliveries kept as history.
2. **Setup → db::setup**
   Branch name + owner name + PIN. Creates the Owner, who is logged in.
3. **Login → db::login**
   Name + PIN; wrong PIN 5 times pauses 30 s. Auto-lock after 5 min idle.
4. **Staff & settings → db::save_user**
   Add staff, set role and PIN, block someone. There must always be one active owner.

Writes: branches · users · settings · audit

Interaction matrix

Which action touches which table. C = create (insert) R = read U = update — every action marked * also writes an audit row.

Action	db.rs function	Who	settings	branches	users	suppliers	products	deliveries	batches	stock_moves	shifts	sales	sale_items	audit
First start / upgrade	`db::open`		C	C	·	C	C	C	C	C	·	C	C	·
Setup (first owner) *	`db::setup`	anyone (once)	R	U	C	·	·	·	·	·	·	·	·	C
Log in *	`db::login`	everyone	R	·	R	·	·	·	·	·	·	·	·	C
Add / edit staff *	`db::save_user`	Owner	·	·	CU	·	·	·	·	·	·	·	·	C
Rename branch *	`db::rename_branch`	Owner	·	U	·	·	·	·	·	·	·	·	·	C
Add / edit medicine *	`db::save_product`	Owner, Manager	·	·	·	·	CU	·	·	·	·	·	·	C
Delete medicine *	`db::delete_product`	Owner, Manager	·	·	·	·	U	·	·	·	·	·	·	C
Add / edit / delete supplier *	`db::save_supplier`	Owner, Manager	·	·	·	CU	·	·	·	·	·	·	·	C
Receive a delivery *	`db::receive`	Owner, Mgr, Pharm.	·	·	·	·	·	C	C	C	·	·	·	C
Write off from a batch *	`db::write_off`	Owner, Manager	·	·	·	·	·	·	R	C	·	·	·	C
Stock count *	`db::count_stock`	Owner, Manager	·	·	·	·	·	·	R	C	·	·	·	C
Open shift *	`db::open_shift`	everyone	·	·	·	·	·	·	·	·	C	·	·	C
Sell (FEFO)	`db::record_sale`	everyone (Rx: not cashier)	·	·	·	·	R	·	R	C	R	C	C	·
Void a sale *	`db::void_sale`	Owner, Manager	·	·	·	·	·	·	·	C	·	U	R	C
Close shift (Z report) *	`db::close_shift`	everyone (own shift)	·	·	·	·	·	·	·	·	U	R	R	C
Reports & CSV export	`db::report / stats`	Owner, Manager	·	·	R	R	R	R	R	R	R	R	R	R

Nothing is ever deleted: 'delete' sets is_active = 0, a void sets status = 'void', and stock changes are new stock_moves rows. Reports read the same tables, filtered by branch and date.